

# WHATNEXT VISION MOTORS

## Salesforce-Based Vehicle Sales, Inventory, Test Drive and Order Management System

### 1. Project Introduction

The WhatNext Vision Motors system is a Salesforce-based vehicle management and CRM application developed to manage the different activities involved in vehicle inventory, customer orders, dealer assignment, test-drive scheduling, and order processing. The main purpose of the system is to bring vehicle management, order handling, dealer coordination, test-drive management, automation, and data processing into a single Salesforce platform.

In a traditional vehicle management process, information may be maintained manually through spreadsheets, emails, phone calls, and separate records. This can make it difficult to track vehicle availability, process customer orders, assign dealers, manage test drives, and update inventory accurately. WhatNext Vision Motors was developed to reduce these manual activities by creating a centralized Salesforce system where vehicle and customer-related information can be stored and managed efficiently.

The system uses Salesforce custom objects such as Vehicle, Vehicle Order, and Test Drive to maintain the required information. These records are connected through Salesforce relationships so that vehicle availability, customer orders, dealer assignments, and test-drive information can be managed in a structured manner. The project also includes Flows, Apex Triggers, Apex Classes, Batch Apex, and Scheduled Apex to automate important business processes.

The project was developed in a Salesforce Developer Edition environment for academic purposes. The implementation demonstrates how Salesforce CRM, database management, automation, and Apex programming can be combined to create a vehicle management solution

### 2. What is the WhatNext Vision Motors System?

**WhatNext Vision Motors** is basically a centralized Salesforce application for managing vehicle inventory and sales-related activities.

The system helps users perform different responsibilities from the same Salesforce application:

- Vehicle records can be maintained with important inventory information.
- Vehicle orders can be created and tracked through different order statuses.
- Dealers can be automatically assigned to vehicle orders.

- Customers can schedule test drives for available vehicles.
- Test-drive information can be maintained and reminder automation can be used.
- Vehicle stock can be updated based on confirmed orders.
- Apex automation can process vehicle orders and perform backend operations.
- Batch and scheduled processing can be used for automated order management.

The system reduces repetitive manual work through Salesforce automation. For example, when a vehicle order is created, the **Vehicle Order Dealer Assignment Flow** can assign the appropriate dealer. The **Vehicle Test Drive Reminder Flow** supports reminder processing for scheduled test drives. Apex Trigger and Apex classes provide additional business logic, while Batch Apex and Scheduled Apex support automated background processing.

The overall implementation therefore combines **CRM, inventory management, order management, workflow automation, Apex programming, and scheduled processing** in one Salesforce application.

### 3. Major Features Covered in the Project

The completed project covers the following major areas:

#### Data Management

- Custom **Vehicle** records for maintaining vehicle information.
- **Vehicle Order** records for managing customer vehicle orders.
- **Test Drive** records for scheduling and tracking vehicle test drives.
- Vehicle inventory and stock information.
- Dealer information associated with vehicle orders.
- Salesforce relationships between the required records.

#### Order Management and Business Logic

- Vehicle order creation and status management.
- Automatic dealer assignment for vehicle orders.
- Confirmation of vehicle orders.
- Automatic vehicle stock updates based on confirmed orders.
- Centralized tracking of vehicle order information.

## Automation

- **Vehicle Order Dealer Assignment Flow** for automatic dealer assignment.
- **Vehicle Test Drive Reminder Flow** for test-drive reminder processing.
- **VehicleOrderTrigger** for handling vehicle-order insert and update events.
- **VehicleOrderTriggerHandler** for implementing trigger-related business logic.
- **VehicleOrderBatch** for processing vehicle orders in batches.
- **VehicleOrderBatchScheduler** for scheduled batch processing.

## Testing and Processing

- Test vehicle record such as **Vision SUV 001**.
- Vehicle order testing using records such as **ORD-002** and **ORD-003**.
- Test-drive record such as **TD-001**.
- Verification of dealer assignment through automation.
- Verification of vehicle stock updates after order confirmation.
- Verification of Flow and Apex components through Salesforce Setup.

## Salesforce Application

- Salesforce-based interface for managing vehicle-related records.
- Centralized access to vehicle, order, and test-drive information.
- Salesforce automation for reducing manual processing.
- Apex-based backend processing for vehicle orders.

These components together make **WhatNext Vision Motors** more than just a collection of Salesforce records. They provide a structured workflow for managing **vehicle inventory, customer orders, dealer assignment, test drives, automated stock updates, and scheduled order processing** within a single Salesforce environment.

# Project Progress, Features & Milestones

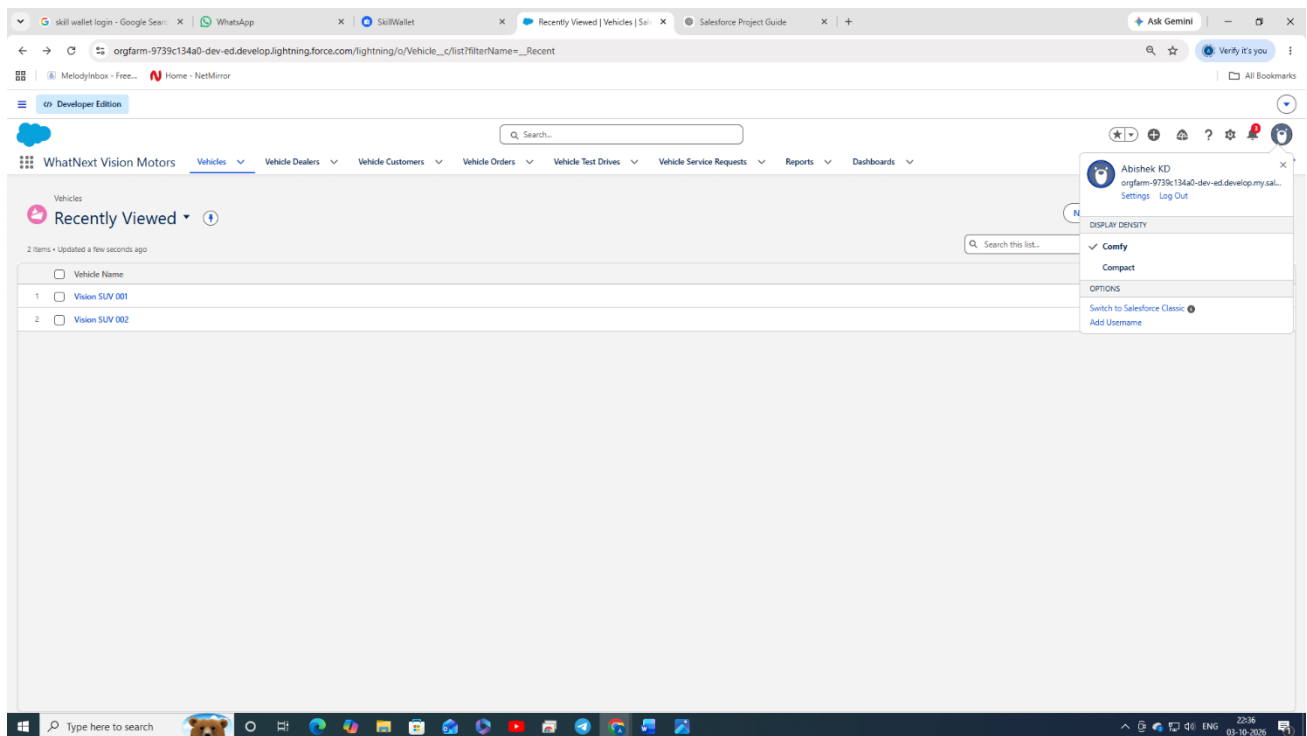
## 1. Vehicle Inventory Management

### Feature Achieved:

Vehicle inventory management was implemented in Salesforce to maintain vehicle information and available stock.

### Milestone Achieved:

Created and maintained the vehicle record Vision SUV 001 with its inventory details.



### Screenshot Explanation:

This screenshot shows the vehicle record created in Salesforce. It demonstrates that vehicle information and stock details can be centrally stored, maintained, and tracked within the WhatNext Vision Motors system.

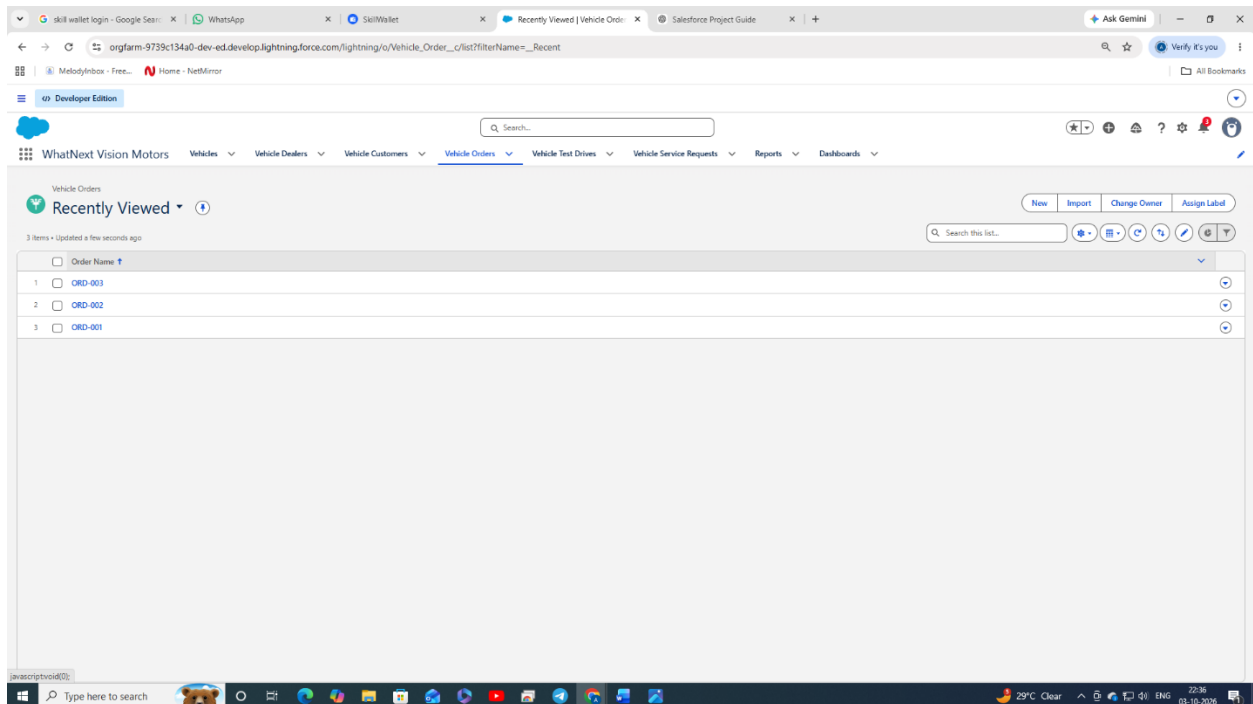
## 2. Vehicle Order Management

### Feature Achieved:

Vehicle order management was implemented to create and track customer vehicle orders.

### Milestone Achieved:

Successfully created and processed vehicle orders including **ORD-002** and **ORD-003**.



### Screenshot Explanation:

This screenshot shows the vehicle order information maintained in Salesforce. It demonstrates that customer vehicle orders can be created, tracked, and managed through the system.

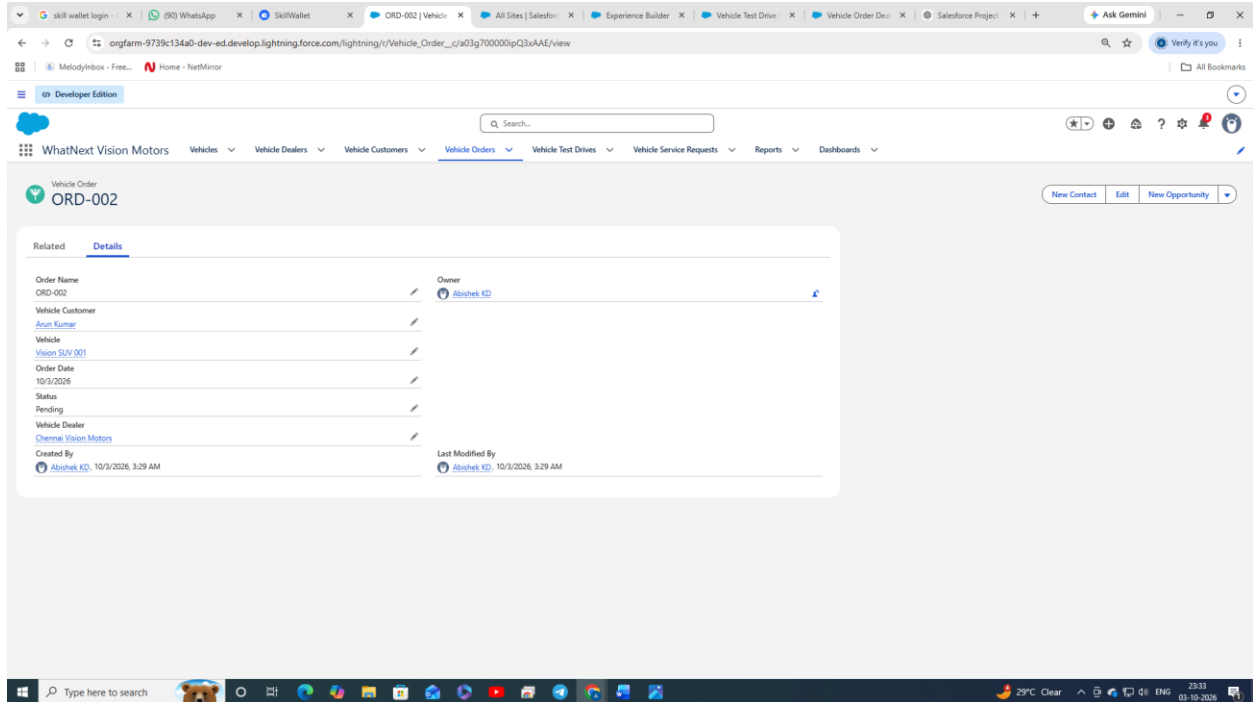
### 3. Automatic Dealer Assignment

#### Feature Achieved:

Automatic dealer assignment was implemented using Salesforce Flow.

#### Milestone Achieved:

Successfully tested the dealer assignment process using **ORD-002**, where the dealer was automatically populated as **Chennai Vision Motors**.



#### Screenshot Explanation:

This screenshot shows the vehicle order after dealer assignment. The populated **Vehicle Dealer** field provides evidence that the system can automatically assign a dealer to a vehicle order, reducing manual processing.

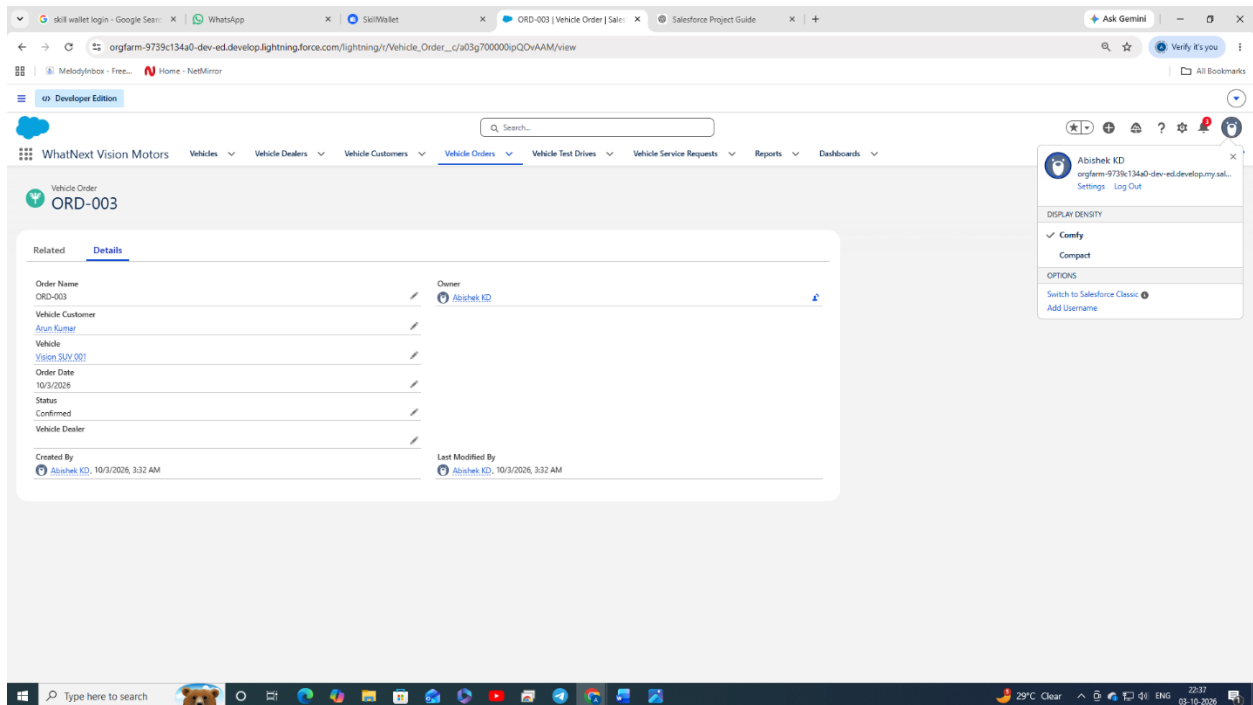
## 4. Vehicle Order Confirmation

### Feature Achieved:

The system supports vehicle order status management and order confirmation.

### Milestone Achieved:

Successfully processed **ORD-003** and changed its status to **Confirmed**.



### Screenshot Explanation:

This screenshot demonstrates the change in vehicle inventory after order processing. It provides evidence that the system can reflect changes in vehicle availability based on confirmed orders.

## 6. Test Drive Management

### Feature Achieved:

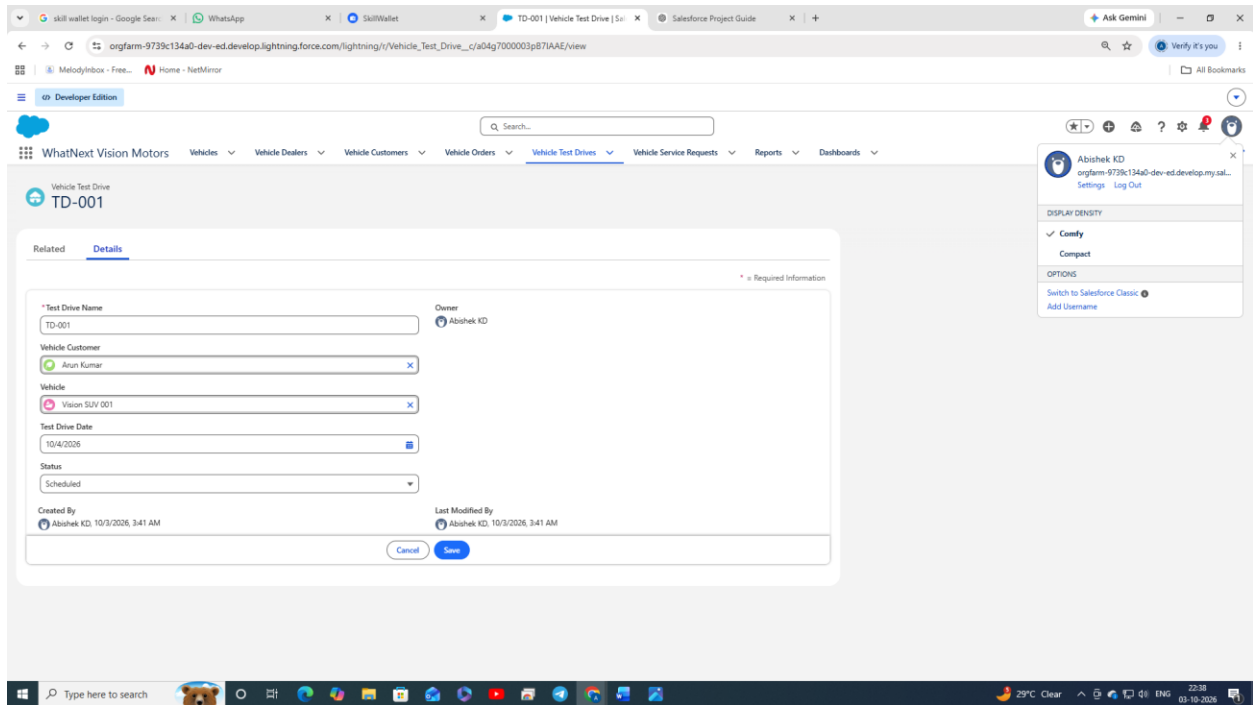
Test-drive scheduling and tracking were implemented in Salesforce.

### Milestone Achieved:

Successfully created **TD-001** for customer **Arun Kumar** and vehicle **Vision SUV 001** with the status set to **Scheduled**.

The screenshot shows a web browser window displaying a Salesforce Lightning page for a 'Vehicle Test Drive' record named 'TD-001'. The browser's address bar shows a URL from a Salesforce development environment. The page header includes the 'WhatNext Vision Motors' logo and a navigation menu with options like 'Vehicles', 'Vehicle Dealers', 'Vehicle Customers', 'Vehicle Orders', 'Vehicle Test Drives' (which is active), 'Vehicle Service Requests', 'Reports', and 'Dashboards'. Below the header, the record details for 'TD-001' are shown in a table-like format. The details include the Test Drive Name, Vehicle Customer (Arun Kumar), Vehicle (Vision SUV 001), Test Drive Date (10/4/2026), Status (Scheduled), Created By (Abhishek KD), and Last Modified By (Abhishek KD). The page also features buttons for 'New Contact', 'Edit', and 'New Opportunity'.

| Vehicle Test Drive |                                 |
|--------------------|---------------------------------|
| TD-001             |                                 |
| Test Drive Name    | TD-001                          |
| Vehicle Customer   | Arun Kumar                      |
| Vehicle            | Vision SUV 001                  |
| Test Drive Date    | 10/4/2026                       |
| Status             | Scheduled                       |
| Created By         | Abhishek KD, 10/3/2026, 3:41 AM |
| Last Modified By   | Abhishek KD, 10/3/2026, 3:41 AM |



### Screenshot Explanation:

This screenshot shows the Test Drive record and its related customer, vehicle, date, and status information. It demonstrates that customer test drives can be scheduled and tracked within the system.

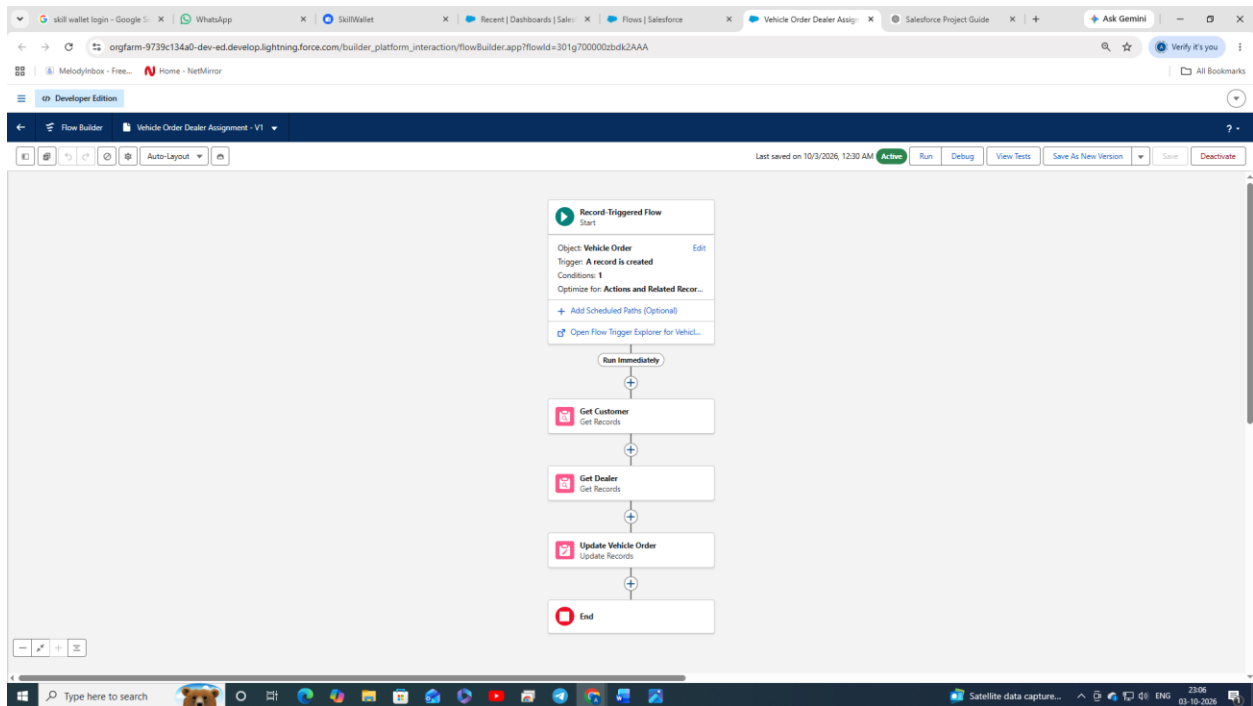
## 7. Vehicle Order Dealer Assignment Flow

Feature Achieved:

Salesforce Flow was implemented to automate dealer assignment for vehicle orders.

Milestone Achieved:

Created and activated the Vehicle Order Dealer Assignment Flow with a record-created trigger and post-save action.



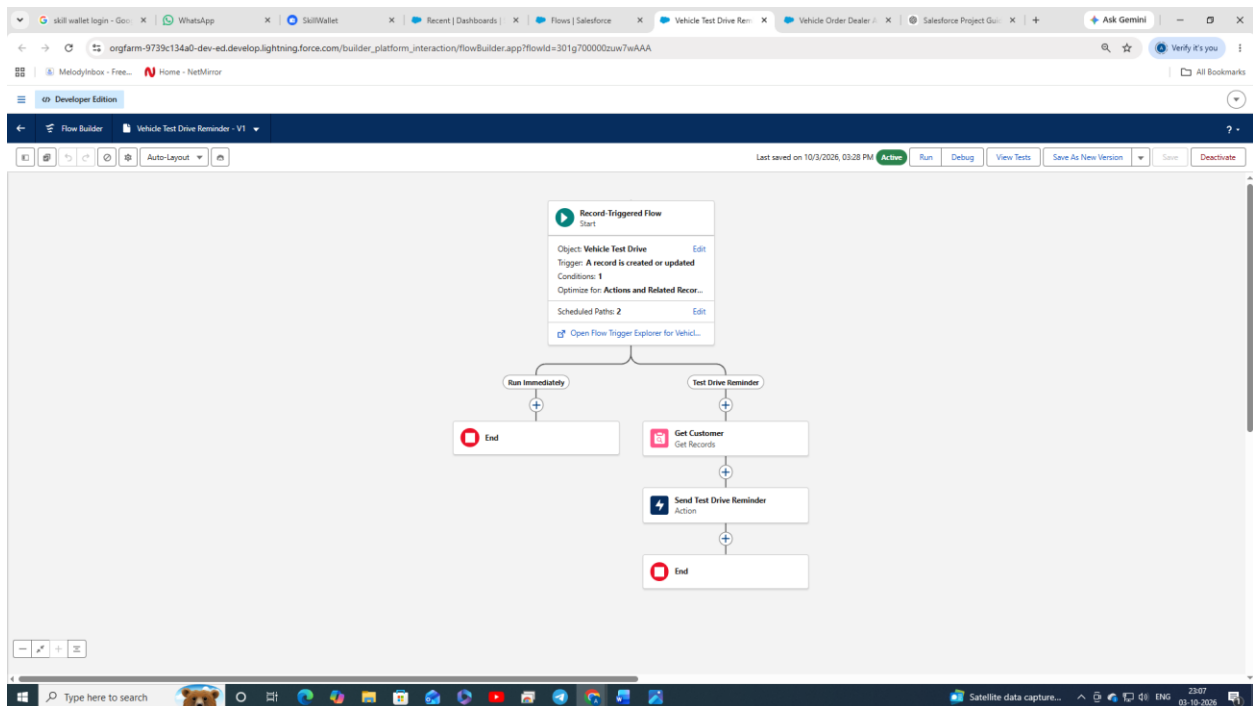
## 8. Vehicle Test Drive Reminder Flow

Feature Achieved:

Automation was implemented to support reminders for scheduled vehicle test drives.

Milestone Achieved:

Created and activated the Vehicle Test Drive Reminder Flow.



### Screenshot Explanation:

This screenshot shows the active Test Drive Reminder Flow. It demonstrates how Salesforce Flow is being used to automate reminder-related processing and reduce the need for manual follow-up.

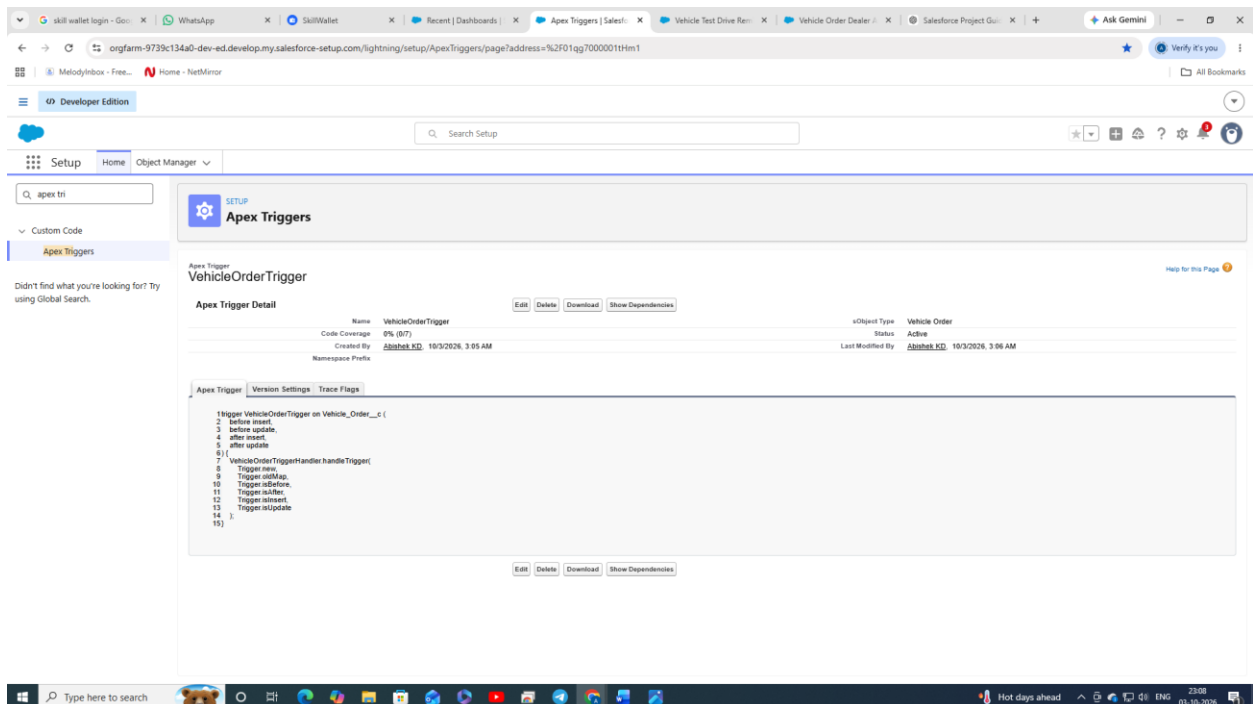
## 9. Apex Trigger Implementation

### Feature Achieved:

The trigger processing logic was separated into an Apex Handler class.

### Milestone Achieved:

Implemented VehicleOrderTriggerHandler to handle the business logic called by the Vehicle Order trigger.



## 11. Batch Apex Processing

### Feature Achieved:

Batch Apex was implemented to support processing of Vehicle Order records in groups.

### Milestone Achieved:

Created the VehicleOrderBatch Apex class for batch processing of vehicle orders.

### Screenshot Explanation:

This screenshot shows the VehicleOrderBatch Apex class. It demonstrates the project's use of Batch Apex for processing multiple vehicle-order records as part of automated backend processing.

The screenshot displays the Salesforce Developer Console interface. The browser address bar shows the URL: `orgfarm-9739c134a0-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ApexClasses/page?address=%2F01pg700000AnZfd`. The page title is "Apex Classes". The left sidebar shows the "Setup" menu with "Apex Classes" selected. The main content area displays the details for the "VehicleOrderBatch" Apex class. The class is created by "Abhishek KD" on 10/5/2026 at 3:07 AM. The status is "Active" with 0% code coverage. The class body is shown in the "Class Body" tab, containing the following code:

```
1 global class VehicleOrderBatch implements Database.Batchable<sObject> {
2     global Database.QueryLocator start(Database.BatchableContext bc) {
3         return Database.queryLocator(
4             SELECT id, Status__c, Vehicle__c
5             FROM Vehicle_Order__c
6             WHERE Status__c = 'Pending'
7         );
8     }
9
10    global void execute(Database.BatchableContext bc) {
11        List<Vehicle_Order__c> orderList;
12        Set<Id> vehicleIds = new Set<Id>();
13        for (Vehicle_Order__c order : orderList) {
14            if (order.Vehicle__c != null) {
15                vehicleIds.add(order.Vehicle__c);
16            }
17        }
18        // Continue with the remaining code from the project here.
19    }
20
21    global void finish(Database.BatchableContext bc) {
22    }
23 }
```

## 12. Scheduled Apex Processing

### Feature Achieved:

Scheduled Apex was implemented to automate background vehicle-order processing.

### Milestone Achieved:

Created VehicleOrderBatchScheduler to schedule the execution of the VehicleOrderBatch process.

The screenshot displays the Salesforce Developer Edition interface. The browser address bar shows the URL: `orgfarm-9739c134a0-dev-ed.develop.my.salesforce-setup.com/lightning/setup/ApexClasses/page?address=%2F01pg700000An5IZ`. The page title is "Apex Classes". The left sidebar shows the "Setup" menu with "Apex Classes" selected. The main content area displays the details for the Apex Class "VehicleOrderBatchScheduler". The class is created by "Abhishek KD" on "10/3/2026, 3:16 AM". The class body is shown in a code editor with the following code:

```
1 global class VehicleOrderBatchScheduler implements Schedulable {
2     global void execute(SchedulableContext sc) {
3         VehicleOrderBatch batchJob = new VehicleOrderBatch();
4         Database.executeBatch(batchJob, 50);
5     }
6 }
```

The right sidebar shows the user profile for "Abhishek KD" with the email "orgfarm-9739c134a0-dev-ed.develop.my.sal...". The sidebar also includes options for "DISPLAY DENSITY" (set to "Compact") and "OPTIONS" (including "Switch to Salesforce Classic" and "Add Username").